



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία 2(2025-2026)

Ονοματεπώνυμο : Ζουμπουρλής Θοδωρής

Αριθμός Μητρώου : 1115202200267

Ονοματεπώνυμο : Τσάκωνας Βασίλης

Αριθμός Μητρώου : 1115202200245

Περιεχόμενα

1	Πληροφορίες για υπολογιστικό σύστημα	2
2	Άσκηση 2.1	2
3	Άσκηση 2.2	5
4	Άσκηση 2.3	10

1 Πληροφορίες για υπολογιστικό σύστημα

Όνομα υπολογιστικού συστήματος: `linux02.di.uoa.gr`

Μοντέλο επεξεργαστή: Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz

Ο επεξεργαστής περιέχει 4 φυσικούς πυρήνες, και κάθε πυρήνας υποστηρίζει 2 νήματα μέσω Hyper-Threading.

Επομένως, διαθέτει 8 λογικούς επεξεργαστές που μπορούν να εκτελούν 8 νήματα ταυτόχρονα.

Έκδοση μεταγλωττιστή: `gcc 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)`

Αφού το `linux02` υποστηρίζει μέχρι 8 νήματα να τρέχουν ταυτόχρονα εμείς τεστάρουμε για 1, 2, 4, 8 και 16 threads.

Επισημάνσεις Στους επιμέρους φακέλους μπορεί κάποιος να βρει τα scripts που χρησιμοποιήθηκαν για τη συλλογή, παραγωγή και ανάλυση των αποτελεσμάτων. Το κάθε script το τρέξαμε 5 φορές και έπειτα συλλέξαμε τα αποτελέσματα και παρουσιάζουμε τον μέσο όρο τους. Επιπλέον, υπάρχουν και τα python scripts που χρησιμοποιήθηκαν για τη παραγωγή των graphs.

2 Άσκηση 2.1

Πρόβλημα και Υλοποίηση

Το πρόβλημα που καλούμαστε να υλοποιήσουμε είναι ο πολλαπλασιασμός πολυωνύμων με `openMP` και σειριακά. Αρχικά υπολογίζουμε το αποτέλεσμα με έναν σειριακό αλγόριθμο και στη συνέχεια ανάλογα με τον αριθμό των νημάτων που έχουμε δώσει ως είσοδο υπολογίζουμε πάλι το ίδιο πολυώνυμο με έναν παράλληλο αλγόριθμο. Κάθε νήμα αναλαμβάνει ένα υποσύνολο των επαναλήψεων του εξωτερικού βρόχου του αλγορίθμου πολλαπλασιασμού. Δεν χρειάζεται κάποιος συγχρονισμός καθώς τα νήματα δε πρόκειται να πάνε να διαβάσουν ή να γράψουν στο ίδιο στοιχείο οπότε δε συμβαίνει κάποιο overlap θέσεων.

Επιπλέον στο `#pragma parallel for` έγινε χρήση `schedule (dynamic)` έπειτα από δοκιμές. Οι δοκιμές αφορούσαν `static` και `dyanmic schedule` και το `dynamic` ήταν πιο γρήγορο λόγω καλύτερου `load-balancing`.

Πίνακας Αποτελεσμάτων

Στο συγκεκριμένο πρόβλημα παρουσιάζονται όλα τα δεκαδικά

Polynomial Degree	10				
Number of Threads	1	2	4	8	16
Initialization Time	0.000002	0.000006	0.000004	0.000004	0.000004
Serial Time	0.000002	0.000002	0.000002	0.000002	0.000002
Parallel Time	-	0.000244	0.000271	0.001713	0.001106
Speedup	1.00	0.012926	0.006516	0.001024	0.001669
Efficiency	1.00	0.006463	0.001629	0.000128	0.000104

Polynomial Degree	100				
Number of Threads	1	2	4	8	16
Initialization Time	0.000016	0.000016	0.000016	0.000054	0.000016
Serial Time	0.000112	0.000112	0.000112	0.000112	0.000112
Parallel Time	-	0.000966	0.000392	0.004129	0.001291
Speedup	1.00	0.1159	0.2857	0.02713	0.08675
Efficiency	1.00	0.0579	0.07143	0.003391	0.00542

Polynomial Degree	1000				
Number of Threads	1	2	4	8	16
Initialization Time	0.000167	0.000148	0.000080	0.000065	0.000077
Serial Time	0.004522	0.004522	0.004522	0.004522	0.004522
Parallel Time	-	0.005063	0.002777	0.002321	0.002405
Speedup	1.00	0.8934	1.6289	1.9483	1.8790
Efficiency	1.00	0.4467	0.4072	0.2435	0.1174

Polynomial Degree	10000				
Number of Threads	1	2	4	8	16
Initialization Time	0.000716	0.000433	0.000398	0.000433	0.000477
Serial Time	0.2492	0.2492	0.2492	0.2492	0.2492
Parallel Time	-	0.236064	0.124223	0.113347	0.114333
Speedup	1.00	1.0558	2.0060	2.1986	2.1796
Efficiency	1.00	0.5279	0.5015	0.2748	0.1362

Polynomial Degree	100000				
Number of Threads	1	2	4	8	16
Initialization Time	0.003440	0.005610	0.004954	0.003377	0.003395
Serial Time	24.198	24.198	24.198	24.198	24.198
Parallel Time	-	23.6773	12.3668	11.263	11.3329
Speedup	1.00	1.0219	1.9570	2.1488	2.1356
Efficiency	1.00	0.5109	0.4893	0.2686	0.1335

Ανάλυση Αποτελεσμάτων

Με βάση τα παραπάνω δεδομένα μπορώ να εξάγω τα εξής συμπεράσματα:

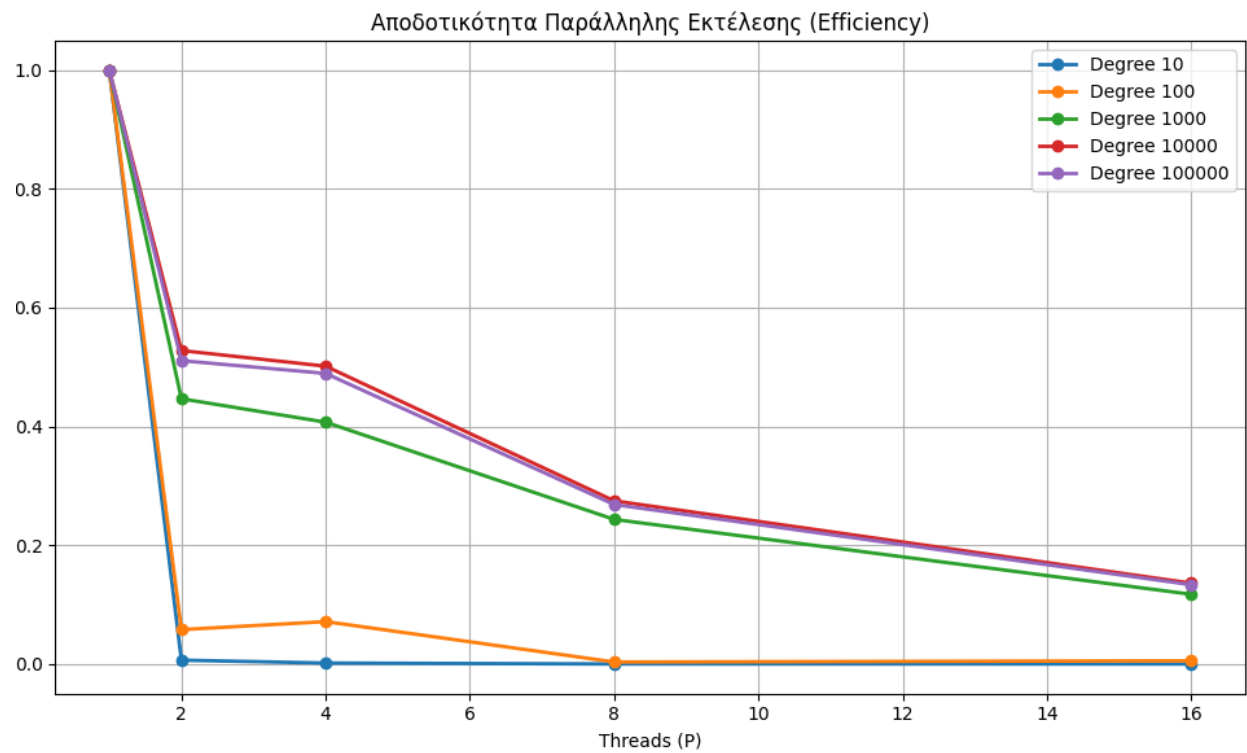
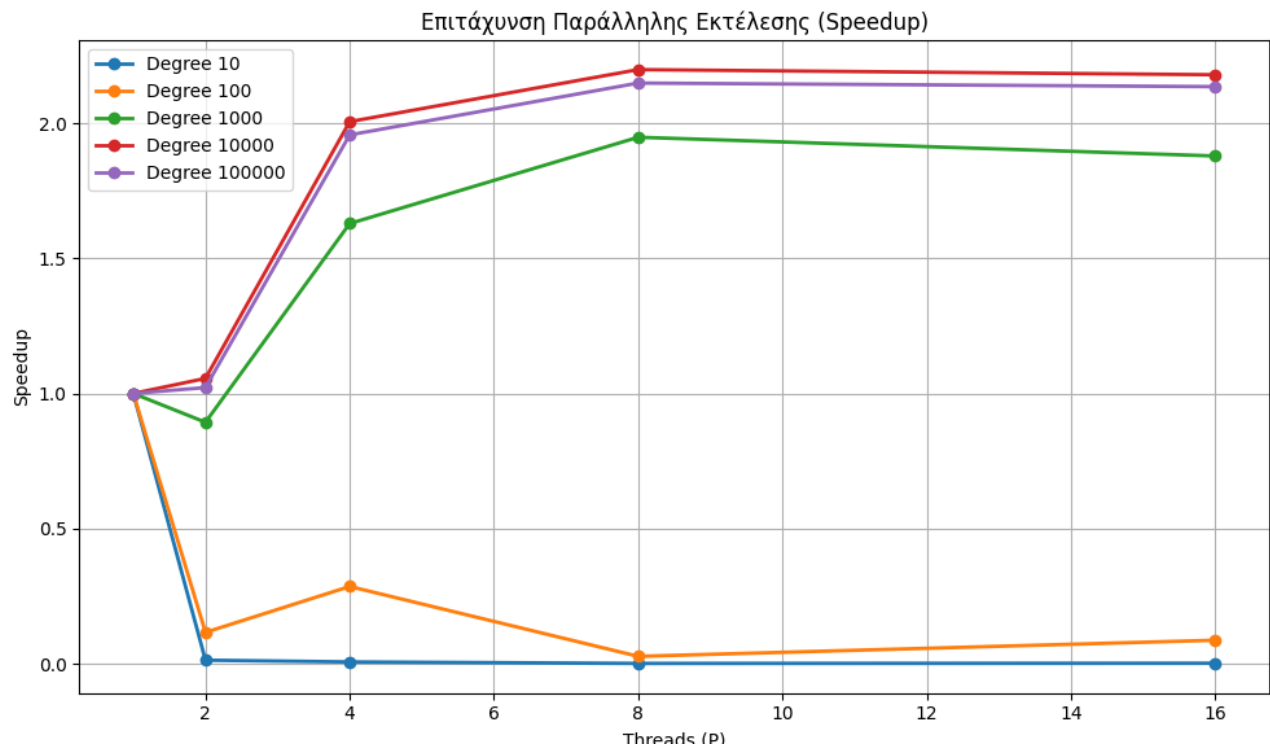
Όσο αυξάνεται ο βαθμός του πολυωνύμου, αυξάνεται και η χρονική διάρκεια του initialization, serial και parallel Time .

Όταν ο βαθμός του πολυωνύμου είναι 10 και 100 ο σειριακός αλγόριθμος είναι ταχύτερος. Αντίθετα για 1000, 10000, 100000 νήματα ο παράλληλος αλγόριθμος είναι ταχύτερος. Αυτό συμβαίνει διότι για μικρά δεδομένα (βαθμός 10 και 100), το overhead (υπολογιστικό κόστος) για τη δημιουργία και τη διαχείριση των νημάτων από τη βιβλιοθήκη OpenMP είναι μεγαλύτερο από τον χρόνο που απαιτείται για τον πολλαπλασιασμό. Στις περιπτώσεις αυτές, η σειριακή εκτέλεση είναι προτιμότερη, καθώς ο επεξεργαστής ολοκληρώνει τις πράξεις πριν καν προλάβει το σύστημα να μοιράσει το φορτίο στα νήματα. Αντίθετα, καθώς ο βαθμός του πολυωνύμου αυξάνεται (10^3 , 10^4 , 10^5), το υπολογιστικό φόρτο γίνεται αρκετά μεγάλο ώστε να αντισταθμίζει το κόστος του παραλληλισμού.

Επιπλέον παρατηρούμε ότι το Speedup και το Efficiency τα εξής:

- **Speedup:** Το Speedup βελτιώνεται σημαντικά όσο αυξάνεται ο βαθμός του πολυωνύμου. Αυτό επιβεβαιώνει ότι ο παραλληλισμός αποδίδει μόνο όταν το υπολογιστικό έργο είναι αρκετά μεγάλο. Τέλος βλέπουμε πως το speedup για 8 και 16 νήματα είναι περίπου το ίδιο καθώς το σύστημα υποστηρίζει παραλληλία 8 νημάτων. Αυτό συμβαίνει καθώς όταν χρησιμοποιούμε 16 νήματα εκτελούνται παράλληλα μόνο τα 8.
- **Μείωση του Efficiency:** Η αποδοτικότητα (Efficiency) τείνει να μειώνεται όσο προσθέτουμε νήματα για ένα σταθερό μέγεθος προβλήματος. Αυτό υποδεικνύει ότι η προσθήκη πολλών νημάτων σε σχετικά μικρά προβλήματα οδηγεί σε πιθανή υποαποσχόληση των πόρων της CPU.

Γραφήματα



Σύγκριση αποτελεσμάτων υλοποίησης με OpenMP και Pthreads

Οι χρόνοι και οι αποδόσεις της υλοποίησης με OpenMP και Pthreads έχουν αρκετές διαφορές. Συγκεκριμένα η υλοποίηση της OpenMP είναι ταχύτερη σε σχέση με αυτή των Pthreads καθώς διαχειρίζεται πιο αποδοτικά τους πόρους του συστήματος μέσω του thread pooling. Στα Pthreads υπάρχει σημαντική καθυστέρηση (overhead) λόγω της συνεχούς δημιουργίας και καταστροφής των νημάτων (pthread_create/join), ενώ η OpenMP διατηρεί μια ομάδα από έτοιμα νήματα που επαναχρησιμοποιούνται. Επιπλέον, οι μηχανισμοί αυτόματου χρονοπρογραμματισμού (scheduling) της OpenMP εξασφαλίζουν καλύτερη εξισορρόπηση φορτίου (load balancing) μεταξύ των πυρήνων. Τέλος, η OpenMP συχνά διαχειρίζεται καλύτερα και την τοπικότητα των δεδομένων στην κρυφή μνήμη (cache), αποφεύγοντας φαινόμενα (false sharing) που συμβαίνουν στα Pthreads και δημιουργούν σημαντικές επιβραδύνσεις.

3 Άσκηση 2.2

Πρόβλημα και Υλοποίηση

Το πρόβλημα αφορά τον πολλαπλασιασμό αραιού πίνακα με διάνυσμα. Η υλοποίηση έχει γίνει με 2 τρόπους. Στον έναν πολλαπλασιάζουμε τη πυκνή μορφή(dense) του πίνακα με το διάνυσμα, ενώ με τον δεύτερο τρόπο κατασκευάζουμε το CSR σχήμα αναπαράστασης και με βάση αυτό πραγματοποιείται ο πολλαπλασιασμός με το διάνυσμα. Και οι 2 υλοποιήσεις εκτελούνται σειριακά αλλά και παράλληλα με τη βοήθεια της βιβλιοθήκης OpenMP. Ακόμα σε κάθε επανάληψη το διάνυσμα του αποτελέσματος είναι το διάνυσμα εισόδου της επόμενης. Έτσι προκύπτουν οι εξής χρόνοι:

Πίνακας Αποτελεσμάτων

Σημείωση: Για τους παρακάτω χρόνους στη CSR αναπαράσταση έχει συμπεριληφθεί και ο χρόνος δημιουργίας της CSR αναπαράστασης.

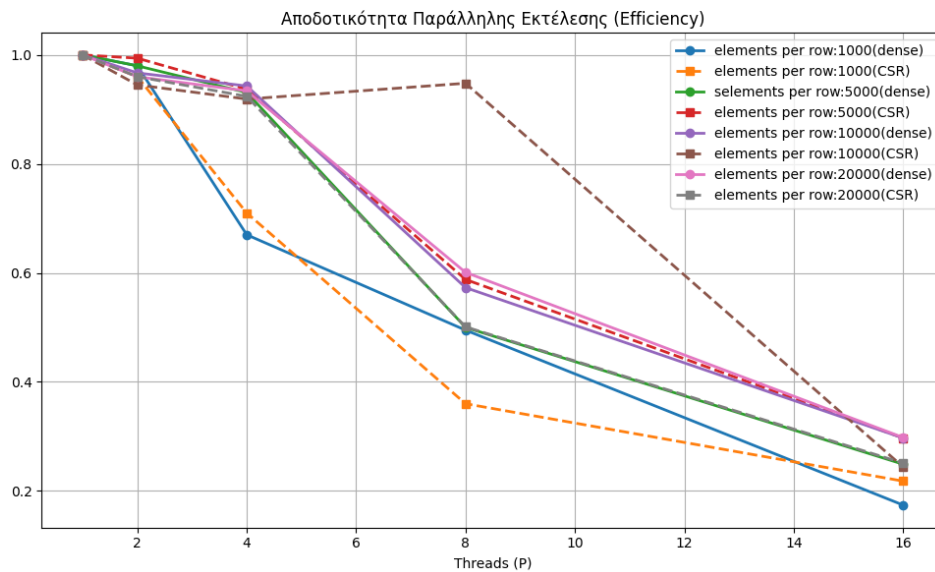
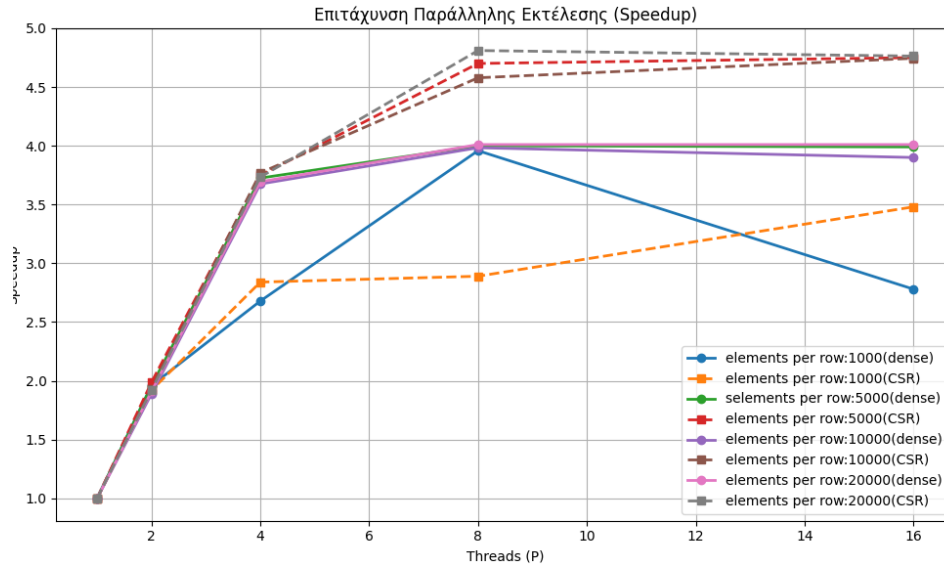
Μεταβολή στο μέγεθος του πίνακα: Για να συγκρίνουμε τον αριθμό των στοιχείων ανά γραμμή κρατήσαμε σταθερό τον αριθμό επαναλήψεων και το ποσοστό μηδενικών στοιχείων, 7 και 50% αντίστοιχα.

Μέγεθος πίνακα	1000									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	0.015	0.025	0.00764	0.013	0.0056	0.0088	0.00379	0.00865	0.00539	0.00718
Speedup	1.00	1.00	1.96	1.92	2.679	2.84	3.958	2.89	2.78	3.48
Efficiency	1.00	1.00	0.98	0.96	0.67	0.71	0.495	0.36	0.174	0.218

Μέγεθος πίνακα	5000									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	0.38	0.62	0.194	0.312	0.102	0.1654	0.095	0.132	0.0953	0.1305
Speedup	1.00	1.00	1.959	1.988	3.725	3.748	4	4.7	3.99	4.75
Efficiency	1.00	1.00	0.98	0.994	0.931	0.937	0.5	0.588	0.249	0.297

Μέγεθος πίνακα	10000									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.47	2.41	0.777	1.247	0.4	0.639	0.369	0.526	0.377	0.508
Speedup	1.00	1.00	1.89	1.933	3.675	3.77	3.983	4.582	3.9	4.744
Efficiency	1.00	1.00	0.945	0.967	0.919	0.943	0.498	0.573	0.244	0.297

Μέγεθος πίνακα	20000									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	5.91	9.67	3.1	5.03	1.6	2.59	1.474	2.01	1.475	2.03
Speedup	1.00	1.00	1.91	1.92	3.694	3.734	4.01	4.81	4.01	4.763
Efficiency	1.00	1.00	0.96	0.96	0.924	0.934	0.501	0.601	0.251	0.298



Από τα παραπάνω μπορούμε να συμπεράνουμε πως όσο μεγαλύτερες είναι οι διαστάσεις του πίνακα τόσο αυξάνεται και το όφελος του παραλληλισμού. (βλ. γραφική παράσταση) Αυτό συμβαίνει καθώς στους μικρούς πίνακες το υπολογιστικό έργο που εκτελεί το κάθε νήμα είναι μικρότερο του overhead των νημάτων. Όταν χρησιμοποιούμε 16 νήματα δε παρατηρείται κάποια βελτίωση καθώς το μηχανήμα μπορεί να εκτελέσει ταυτόχρονα μόνο 8. Επιπλέον, για όλα τα μεγέθη πίνακα που εξετάστηκαν, η πυκνή (dense) μορφή αναπαράστασης παρουσιάζει μικρότερο απόλυτο χρόνο εκτέλεσης σε σύγκριση με την CSR. Παρ' όλα αυτά, η CSR αναπαράσταση εμφανίζει καλύτερο speedup όσο αυξάνεται ο αριθμός των νημάτων.

Μεταβολή στον αριθμό των επαναλήψεων

Για να συγκρίνουμε τον αριθμό των επαναλήψεων ανά γραμμή κρατήσαμε σταθερό τον αριθμό των στοιχείων ανά γραμμή και το ποσοστό μηδενικών στοιχείων, 10000 και 50% αντίστοιχα.

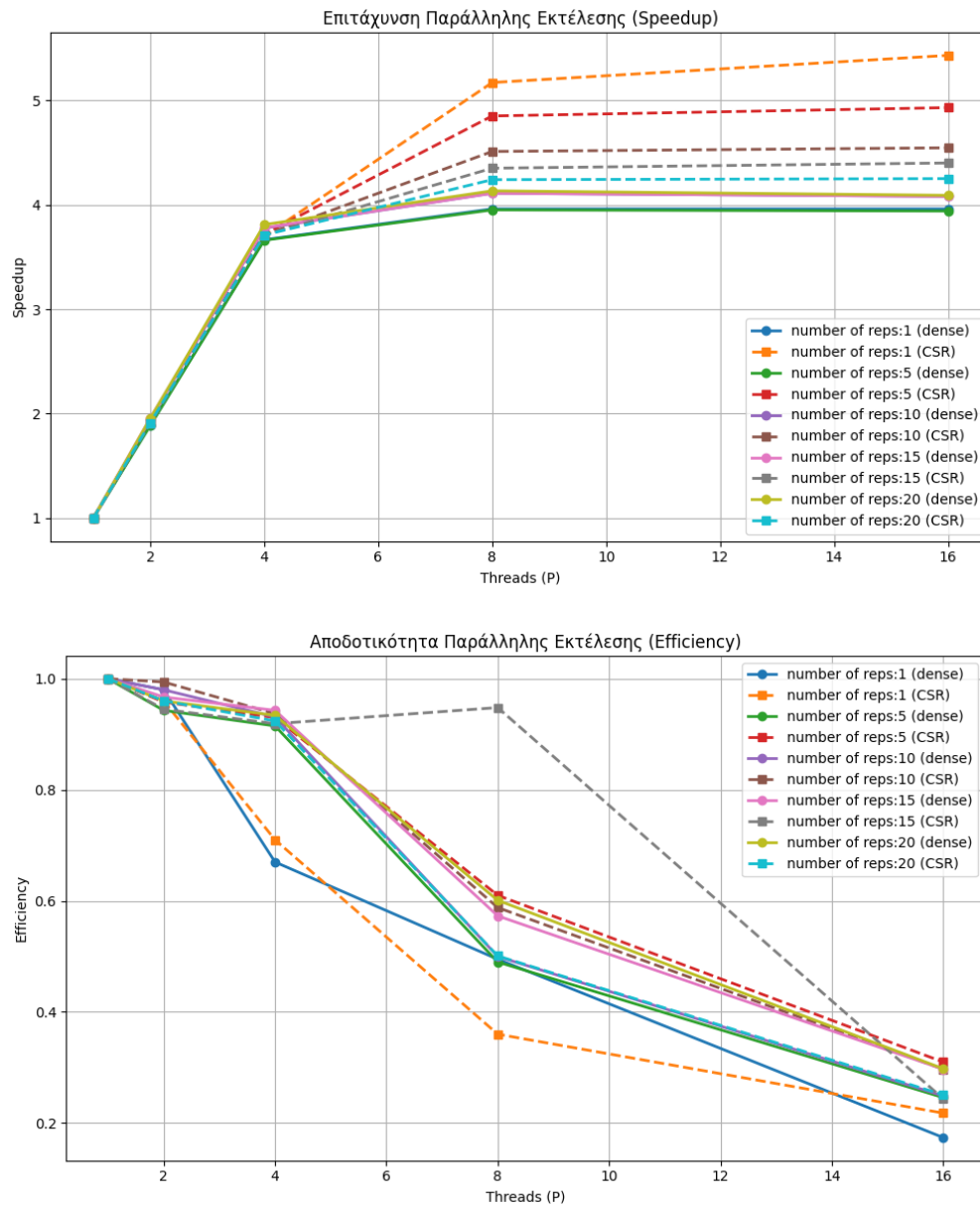
Αριθμός επαναλήψεων	1									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	0.21	1.629	0.111	0.84	0.0573	0.44	0.053	0.315	0.053	0.3
Speedup	1.00	1.00	1.892	1.94	3.665	3.7	3.96	5.17	3.96	5.43
Efficiency	1.00	1.00	0.946	0.97	0.92	0.925	0.495	0.646	0.248	0.34

Αριθμός επαναλήψεων	5									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.047	2.15	0.555	1.122	0.286	0.578	0.265	0.443	0.266	0.436
Speedup	1.00	1.00	1.886	1.916	3.66	3.72	3.951	4.85	3.94	4.93
Efficiency	1.00	1.00	0.943	0.958	0.915	0.93	0.49	0.61	0.246	0.31

Αριθμός επαναλήψεων	10									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	2.16	2.8	1.11	1.467	0.572	0.755	0.526	0.621	0.53	0.616
Speedup	1.00	1.00	1.946	1.91	3.776	3.71	4.11	4.51	4.08	4.545
Efficiency	1.00	1.00	0.973	0.955	0.944	0.9275	0.514	0.564	0.255	0.28

Αριθμός επαναλήψεων	15									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	3.25	3.46	1.66	1.816	0.86	0.932	0.79	0.795	0.797	0.786
Speedup	1.00	1.00	1.956	1.91	3.78	3.71	4.11	4.35	4.08	4.4
Efficiency	1.00	1.00	0.978	0.955	0.945	0.93	0.514	0.544	0.255	0.275

Αριθμός επαναλήψεων	20									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	4.34	4.11	2.22	2.15	1.14	1.109	1.05	0.97	1.06	0.967
Speedup	1.00	1.00	1.955	1.912	3.81	3.71	4.133	4.24	4.09	4.25
Efficiency	1.00	1.00	0.977	0.956	0.953	0.93	0.517	0.53	0.256	0.266



Από τα παραπάνω μπορούμε να συμπεράνουμε πως η CSR αναπαράσταση είναι πιο γρήγορη σε σχέση με την πυκνή μορφή όταν έχουμε μεγαλύτερο αριθμό επαναλήψεων. Αυτό συμβαίνει διότι το μεγαλύτερο μέρος του χρόνου της CSR αναπαράστασης το καταναλώνει η δημιουργία του CSR πίνακα. Επομένως όταν υπάρχουν πολλές επαναλήψεις η CSR αναπαράσταση δημιουργείται μόνο μια φορά και μετά με βάση αυτή πραγματοποιούνται όλες οι επαναλήψεις. Η αύξηση του αριθμού των επαναλήψεων ενισχύει την απόδοση της παραλληλίας, καθώς τα νήματα παραμένουν ενεργά για μεγαλύτερο χρονικό διάστημα και το υπολογιστικό έργο καταφέρνει να είναι μεγαλύτερο του overhead της λειτουργίας τους. Έτσι, παρατηρείται καλύτερο speedup, ιδιαίτερα στην CSR αναπαράσταση, όπου το υπολογιστικό έργο κατανέμεται πιο αποτελεσματικά μεταξύ των νημάτων. Όπως και πριν, παρατηρείται πως όταν αυξάνεται ο αριθμός των νημάτων από 8 σε 16 τότε δεν υπάρχει κάποια πραγματική βελτίωση και αυτό οφείλεται στο ότι το μηχανήμα μπορεί να εκτελέσει ταυτόχρονα μέχρι 8 νήματα. Τέλος, παρατηρείται πως το speedup και η απόδοση του πολλαπλασιασμού με CSR είναι καλύτερη σε σχέση με την πυκνή μορφή του πίνακα ανεξαρτήτως του αριθμού των επαναλήψεων.

Μεταβολή στο ποσοστό των μηδενικών στοιχείων

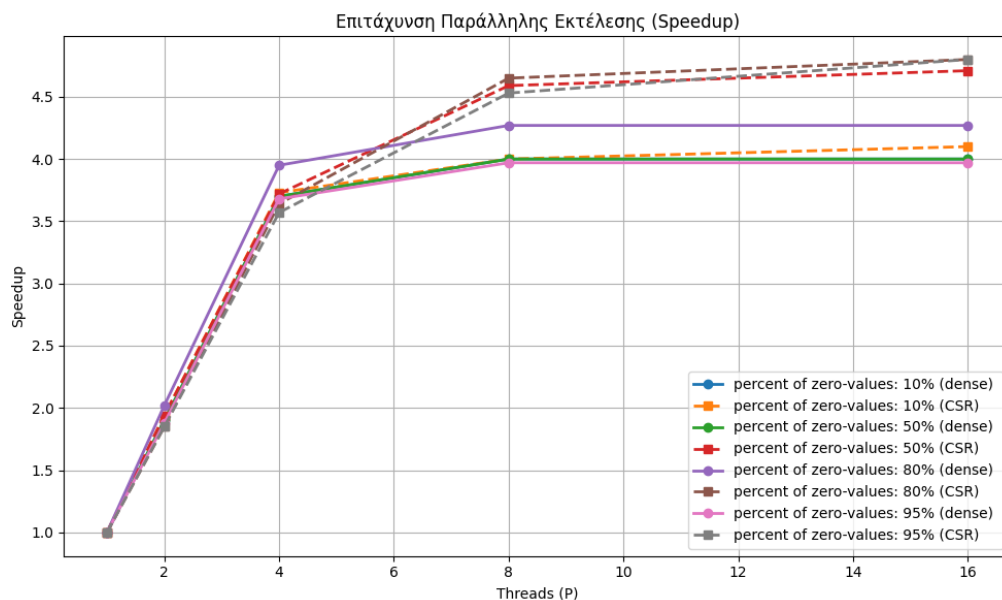
Για να συγκρίνουμε το ποσοστό μηδενικών στοιχείων του πίνακα κρατήσαμε σταθερό τον αριθμό των στοιχείων ανά γραμμή και τον αριθμό των επαναλήψεων, 10000 και 7 αντίστοιχα.

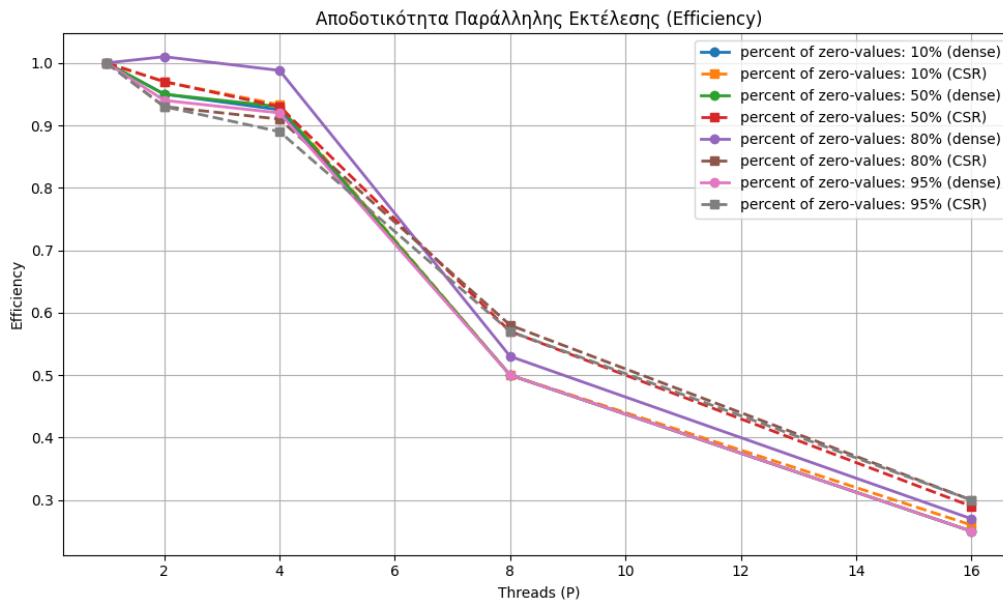
Ποσοστό μηδενικών στοιχείων	10%									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.48	2.65	0.78	1.364	0.4	0.71	0.37	0.662	0.37	0.647
Speedup	1.00	1.00	1.9	1.94	3.7	3.73	4.00	4.00	4.00	4.1
Efficiency	1.00	1.00	0.95	0.97	0.925	0.933	0.5	0.5	0.25	0.26

Ποσοστό μηδενικών στοιχείων	50%									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.48	2.41	0.78	1.24	0.4	0.648	0.37	0.5245	0.37	0.512
Speedup	1.00	1.00	1.9	1.94	3.7	3.72	4.00	4.59	4.00	4.71
Efficiency	1.00	1.00	0.95	0.97	0.93	0.93	0.5	0.57	0.25	0.29

Ποσοστό μηδενικών στοιχείων	80%									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.58	1.2	0.78	0.648	0.4	0.33	0.37	0.258	0.37	0.25
Speedup	1.00	1.00	2.02	1.85	3.95	3.64	4.27	4.65	4.27	4.8
Efficiency	1.00	1.00	1.01	0.93	0.988	0.91	0.53	0.58	0.27	0.3

Ποσοστό μηδενικών στοιχείων	95%									
Αριθμός νημάτων	1		2		4		8		16	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος (sec)	1.47	0.571	0.78	0.309	0.4	0.16	0.37	0.126	0.37	0.119
Speedup	1.00	1.00	1.88	1.85	3.68	3.57	3.97	4.53	3.97	4.8
Efficiency	1.00	1.00	0.94	0.93	0.92	0.89	0.50	0.57	0.25	0.3





Από τα παραπάνω συμπεραίνουμε πως ο πολλαπλασιασμός χρησιμοποιώντας την CSR αναπαράσταση είναι πιο γρήγορος στη περίπτωση όπου ο πίνακας είναι πιο αραιός, δηλ. το ποσοστό των μηδενικών στοιχείων είναι μεγαλύτερο. Αυτό συμβαίνει γιατί στην CSR αναπαράσταση ο πολλαπλασιασμός πραγματοποιείται μόνο για μη-μηδενικά στοιχεία ενώ στη πυκνή μορφή πολλαπλασιάζονται όλα τα στοιχεία του πίνακα. Όπως και πριν, παρατηρείται πως όταν αυξάνεται ο αριθμός των νημάτων από 8 σε 16 τότε δεν υπάρχει κάποια πραγματική βελτίωση και αυτό οφείλεται στο ότι το μηχανήμα μπορεί να εκτελέσει ταυτόχρονα μέχρι 8 νήματα.

Συμπέρασμα

Το τελικό συμπέρασμα είναι πως ο πολλαπλασιασμός αραιού πίνακα με διάνυσμα είναι πιο γρήγορος με την CSR αναπαράσταση μόνο στις περιπτώσεις όπου ο αριθμός των επαναλήψεων είναι μεγάλος και όταν το ποσοστό των μηδενικών στοιχείων είναι αρκετά μεγάλο. Σε όλες τις άλλες περιπτώσεις η πράξη επιτυγχάνεται πιο γρήγορα όταν γίνεται χρήση της πυκνής μορφής του πίνακα. Τέλος, η CSR αναπαράσταση αξιοποιεί την παραλληλία σε μεγαλύτερο βαθμό (καλύτερο speedUp και efficiency ανεξαρτήτως παραμέτρων).

4 Άσκηση 2.3

Πρόβλημα και Υλοποίηση

Το πρόβλημα που καλούμαστε να υλοποιήσουμε είναι η ταξινόμηση πίνακα (merge sort) με openMP και σειριακά. Οι 2 προσεγγίσεις τρέχουν ανεξάρτητα, ανάλογα το όρισμα που δίνεται από την γραμμή εντολών (serial or parallel). Η τεχνική του σειριακού αλγορίθμου σπάει το πρόβλημα σε υποπροβλήματα και μετά τα συγχωνεύει (divide and conquer). Με παρόμοια λογική υλοποιείται και ο παράλληλος αλγόριθμος με την μόνη διαφορά ότι χρησιμοποιεί το μοντέλο Task Parallelism. Συγκεκριμένα έχει εφαρμοστεί η παρακάτω λογική :

Δημιουργία Tasks: Κάθε αναδρομική κλήση ορίζεται ως μια ανεξάρτητη μονάδα εργασίας (task) μέσω της οδηγίας `pragma omp task`

Διαχείριση από Threads: Τα διαθέσιμα νήματα (threads) του συστήματος αναλαμβάνουν την εκτέλεση των διαθέσιμων tasks από μια κοινή ουρά, επιτυγχάνοντας δυναμική εξισορρόπηση φορτίου (dynamic load balancing).

Συγχρονισμός: Η οδηγία `pragma omp taskwait` διασφαλίζει ότι η συγχώνευση (merge) θα ξεκινήσει μόνο αφού ολοκληρωθεί η ταξινόμηση των δύο αντίστοιχων υποπινάκων.

Βελτιστοποίηση με Cutoff: Για να αποφευχθεί το υπερβολικό overhead από τη δημιουργία πολύ μικρών tasks, χρησιμοποιείται η παράμετρος TASK_THRESHOLD = 10000. Όταν το μέγεθος του υποπίνακα πέσει κάτω από το όριο, δεν δημιουργούνται νέα tasks και η αναδρομή συνεχίζεται σειριακά στο ίδιο νήμα.

Πίνακας Αποτελεσμάτων

Table size	10^2				
Number of Threads	1	2	4	8	16
Serial Time	0.000047	0.000047	0.000047	0.000047	0.000047
Parallel Time	-	0.000080	0.000143	0.000264	0.000981
Speedup	1.00	0.586	0.3287	0.17803	0.04791
Efficiency	1.00	0.293	0.082	0.0223	0.00294

Table size	10^3				
S Number of Threads	1	2	4	8	16
Serial Time	0.000187	0.000187	0.000187	0.000187	0.000187
Parallel Time	-	0.000655	0.000806	0.00286	0.00146
Speedup	1.00	0.286	0.232	0.0654	0.128
Efficiency	1.00	0.143	0.058	0.00818	0.008

Table size	10^4				
Number of Threads	1	2	4	8	16
Serial Time	0.006323	0.006323	0.006323	0.006323	0.006323
Parallel Time	-	0.00255	0.00452	0.0061	0.00466
Speedup	1.00	1.1035	1.05	0.744	0.894
Efficiency	1.00	0.552	0.263	0.093	0.0559

Table size	10^5				
Number of Threads	1	2	4	8	16
Serial Time	0.042876	0.042876	0.042876	0.042876	0.042876
Parallel Time	-	0.01665	0.0134	0.0148	0.02263
Speedup	1.00	2.575	3.2	2.897	1.89
Efficiency	1.00	1.288	0.8	0.362	0.118

Table size	10^6				
Number of Threads	1	2	4	8	16
Serial Time	0.20897	0.20897	0.20897	0.20897	0.20897
Parallel Time	-	0.133	0.1051	0.0965	0.0657
Speedup	1.00	1.57	1.99	2.166	3.18
Efficiency	1.00	0.785	0.498	0.271	0.199

Table size	10^7				
Number of Threads	1	2	4	8	16
Serial Time	2.2821	2.2821	2.2821	2.2821	2.2821
Parallel Time	-	1.375	1.074	0.8209	0.6725
Speedup	1.00	1.66	2.125	2.78	3.39
Efficiency	1.00	0.83	0.531	0.348	0.212

Table size	10^8				
Number of Threads	1	2	4	8	16
Serial Time	25.2381	25.2381	25.2381	25.2381	25.2381
Parallel Time	-	14.913	12.217	8.3813	7.033
Speedup	1.00	1.69	2.067	3.01	3.589
Efficiency	1.00	0.845	0.515	0.377	0.2243

Ανάλυση Αποτελεσμάτων

Με βάση τα παραπάνω αποτελέσματα μπορούμε να εξάγουμε τα εξής συμπεράσματα :

Για μικρά μεγέθη πίνακα ($n=100, 1000$), παρατηρούμε ότι ο παράλληλος χρόνος εκτέλεσης είναι πιο αργός σε σχέση με τον σειριακό. Αυτό συμβαίνει γιατί ο χρόνος που απαιτείται για τη δημιουργία των OpenMP tasks, τη διαχείριση της ουράς εργασιών και τον συγχρονισμό των νημάτων είναι πολύ μεγαλύτερος από τον χρόνο που χρειάζεται η CPU για να ταξινομήσει σειριακά μερικά στοιχεία.

Η πραγματική ισχύς του παραλληλισμού αρχίζει να φαίνεται για πίνακα μεγέθους 100000 στοιχείων και άνω. Στο μέγιστο μέγεθος ($n=100.000.000$) . Παρατηρούμε ότι όσο αυξάνεται ο όγκος των δεδομένων, ο αλγόριθμος λειτουργεί καλύτερα, βελτιώνοντας σταδιακά την ταχύτητα εκτέλεσης άλλα και την διαχείριση των νημάτων.

Παρά την αύξηση της ταχύτητας, η αποδοτικότητα (efficiency) μειώνεται σημαντικά όσο αυξάνονται τα νήματα, λόγω Νόμο του Amdahl.

Τέλος Ο αλγόριθμος χρησιμοποιεί ένα TASK_THRESHOLD για να περιορίσει την δημιουργία tasks. Χωρίς την τεχνική αυτή το σύστημα θα κατέρρεε από το τεράστιο πλήθος των tasks. Η μετατροπή του αλγορίθμου σε σειριακό για μικρά τμήματα πίνακα επιτρέπει στα νήματα να εκτελούν τις λειτουργίες τους με πολύ μικρότερο overhead.

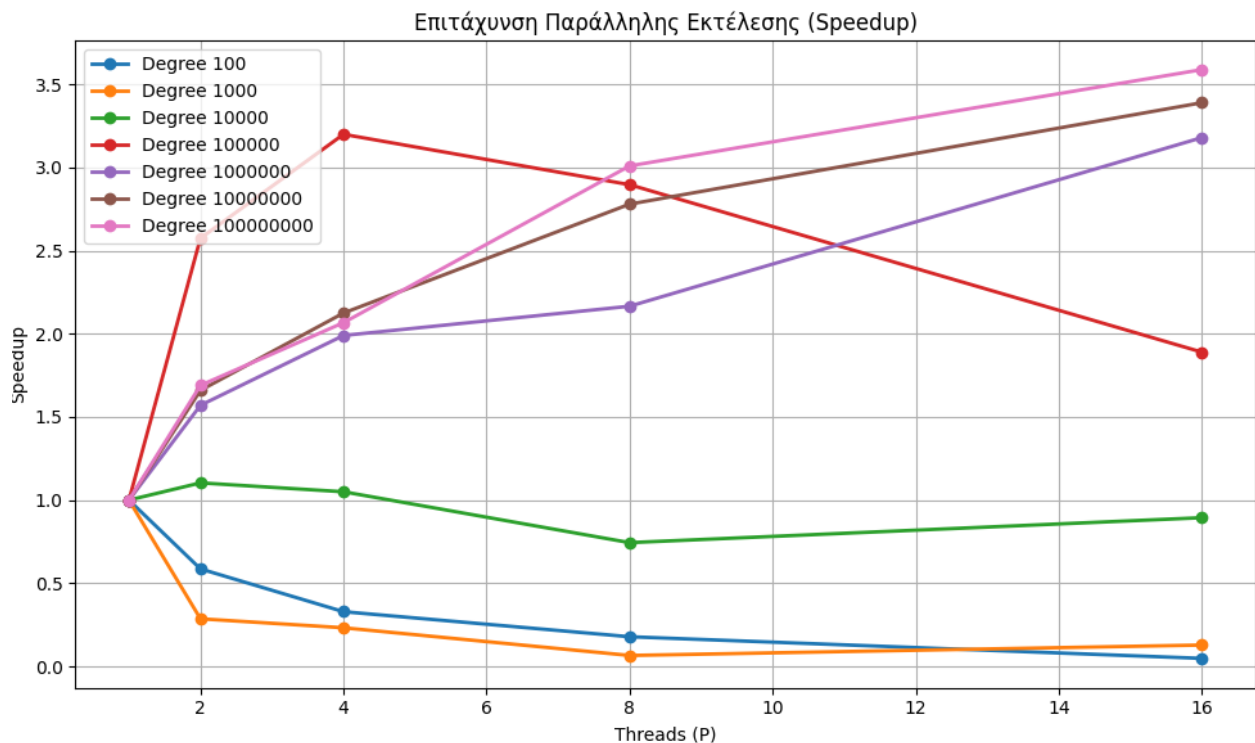
Σημείωση για TASK_THRESHOLD

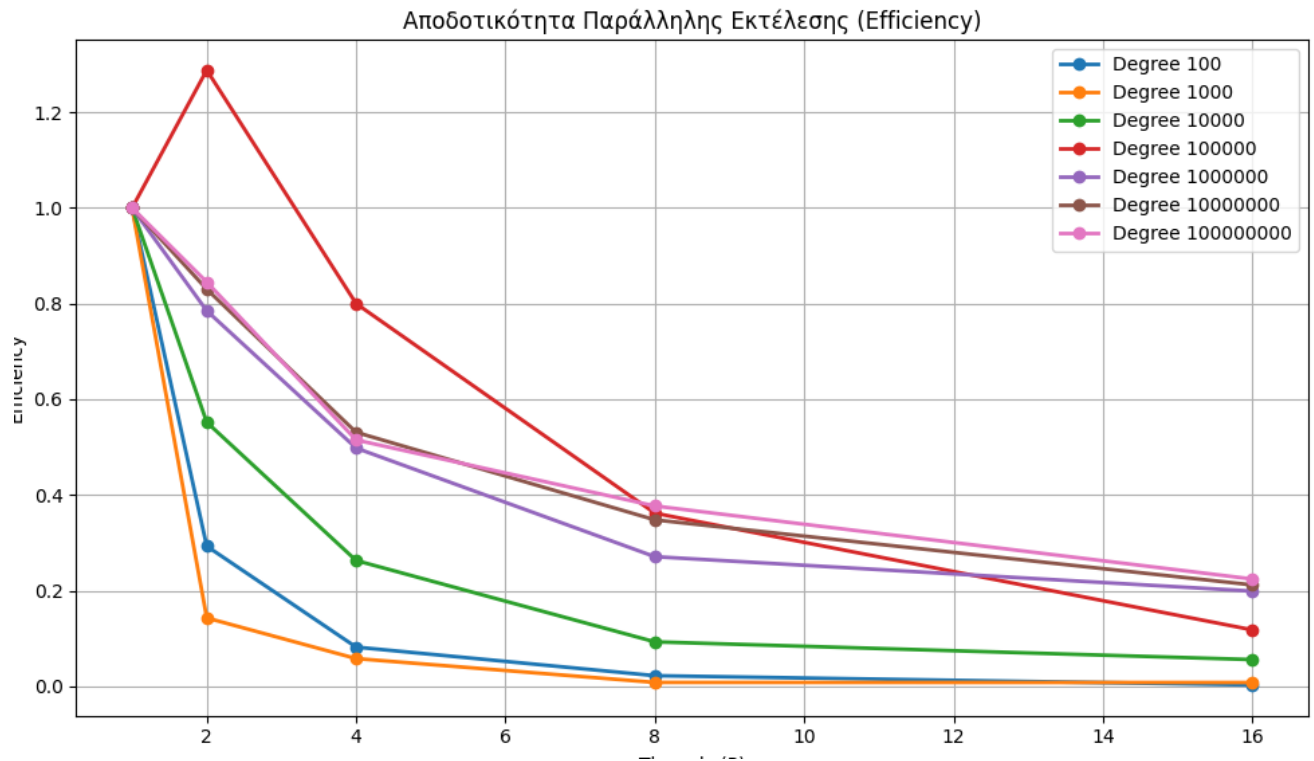
Αν το TASK_THRESHOLD ήταν πολύ μικρό θα είχαμε υπερβολικό overhead (δημιουργία tasks), task saturation (δημιουργία πολλών tasks). Επιπλέον, το speedup και το efficiency θα έπεφταν κατακόρυφα.

Από την άλλη αν το TASK_THRESHOLD ήταν πολύ μεγάλο θα συνέβαινε υποαπασχόληση των νημάτων, το load balancing θα ήταν μη-αποδοτικό και ο χρόνος εκτέλεσης θα πλησίαζε τον σειριακό χρόνο.

Υστερα από αρκετές δοκιμές καταλήξαμε πως ο βέλτιστος χρόνος επιτυγχάνεται για TASK_THRESHOLD = 10000.

Γραφήματα





Στα παραπάνω γραφήματα παρατηρούμε πως για $n = 10^5$ το efficiency παρουσιάζει τιμές μεγαλύτερες της μονάδας για μικρό αριθμό νημάτων. Αυτό οφείλονται σε superlinear speedup, πιθανό αποτέλεσμα βελτιωμένης cache locality και καλύτερης εκμετάλλευσης της μνήμης, χωρίς να παραβιάζεται η θεωρητική έννοια της αποδοτικότητας.